O datapath do processador (sem pipeline) funciona da seguinte maneira:

PC aponta para a Dual Port RAM. O valor do PC é controlado por 2 MUX: um para a instrução ji e outro para brzr, controlados pelos sinais PCSumSrc e Branch, respectivamente. Na subida do clock, é lido o dado contido no endereço apontado (data), enquanto na descida é lida a instrução (instr). Ao longo da execução do trabalho, percebeu-se que essa ordem de leitura não funcionava adequadamente, especialmente para a instrução brzr. Logo, para essa implementação, o PC, a Memória de Instruções e o Banco de Registradores recebem o clock invertido por uma porta NOT.

A Dual Port RAM possui 5 entradas:

- addr: endereço de leitura ou escrita de dados, recebe como entrada o registrador rb.
- pc: posição atual do PC.
- d: dado a ser escrito na memória, recebe como entrada o registrador ra.
- w: sinal de escrita na memória, recebe uma porta AND para o opcode da instrução st (0011)
- clock.

Em seguida, a instrução passa por um splitter, que a divide em opcode e dados. O opcode é enviado diretamente para a Unidade de Controle, enquanto os últimos 4 bits de dados são enviados ao Banco de Registradores e à ULA. Os 4 bits que são enviados à ULA passam por um extensor de bits, que os converte em 8 bits.

A Unidade de Controle foi construída a partir de uma tabela-verdade com os opcodes e os respectivos sinais que devem ser gerados:

opcode[3..0]	ALUOp[3..0]	ALUSrc	Branch	RegWr	MemtoReg	PcSumSrc	Incr
0000	0001	0	1	0	0	0	0
0001	0000	0	0	0	0	1	0
0010	0000	0	0	1	1	0	0
0011	0000	0	0	0	0	0	0
0100	0000	1	0	1	0	0	0
0101	1000	1	0	1	0	0	0
0110	1001	0	0	1	0	0	0
0111	0000	0	0	1	0	0	1
1000	0010	0	0	1	0	0	0
1001	0011	0	0	1	0	0	0
1010	0100	0	0	1	0	0	0
1011	0101	0	0	1	0	0	0
1100	0000	0	0	1	0	0	0
1101	0001	0	0	1	0	0	0
1110	0110	0	0	1	0	0	0
1111	0111	0	0	1	0	0	0

- ALUOp: define a operação a ser executada pela ULA;
- ALUSrc: controla um MUX que interpreta instruções R e I. Ele seleciona registrador rb quando ALUSrc = 0 e imediato quando ALUSrc = 1 para uma das entradas da ULA;
- Branch: define quando ocorre um salto no endereço apontado pelo PC;
- RegWr: define quando deve haver escrita de dados no banco de registradores;

- MemtoReg: controla um MUX que interpreta se o dado a ser escrito no banco de registradores deve vir da memória ou da ULA. Ele seleciona a saída da ULA quando MemtoReg = 0 e dado da memória quando MemtoReg = 1;
- PCSumSrc: controla um MUX que define o valor a ser somado ao PC, utilizado para a instrução ji. Ele seleciona uma constante 1 (funcionamento normal do PC) quando PCSumSrc = 0 e o imediato que vem da instrução quando PCSumSrc = 1;
- Incr: controla um MUX, presente na ULA, que define quando o valor armazenado no registrador ra deve ser incrementado. Ele seleciona o registrador rb quando Incr = 0 e uma constante 1 quando Incr = 1. Isso é utilizado para a instrução incr, que foi criada para essa implementação.

As instruções executadas pelos opcodes seguem o padrão presente no conjunto de instruções REDUX-V fornecido previamente. As três novas instruções escolhidas foram:
- muli (opcode 0101, tipo I): multiplicação por imediato, de forma análoga à soma feita em addi, isto é, operação realizada diretamente sobre o registrador r0. Faz-se útil especialmente para o cálculo de endereços de salto ou load/store. Isso porque, para essas operações, acabava sendo necessário repetir sucessivamente a instrução add r0, r0, até se obter o valor desejado.
- mv (opcode 0110, tipo R): move diretamente o conteúdo de um registrador para outro, de forma análoga à instrução load, que simplesmente substitui o conteúdo anterior pelo novo. Essa instrução se faz útil nessa implementação uma vez que remove a necessidade de se executar 4 instruções para isso: zerar r0 -> adicionar imm em r0 -> zerar o registrador destino rd -> add r0, rd.
- incr (opcode 0111, tipo R): incrementa o valor armazenado no registrador ra. Sua utilidade se deve ao fato de essa ser uma operação muito frequente em loops, por exemplo. Desse modo, ela simplifica a incrementação do iterador em um loop, que anteriormente exigia que ocorressem as operações: zerar r0 -> adicionar 1 em r0 -> add r0, iterador.

O Banco de Registradores recebe como entrada os 4 bits menos significativos da instrução divididos em ra e rb. Além disso, é influenciado pelos sinais de controle ALUSrc, RegWr e MemtoReg. As saídas são os valores armazenados em ra e rb.

Aqui, o ALUSrc se fez necessário para definir o registrador de destino para instruções do tipo I, uma vez que elas são efetuadas diretamente sobre o registrador r0. Então, esse sinal controla um MUX que define rd = ra, quando ALUSrc = 0 e rd = r0, quando ALUSrc = 1.

O Banco também possui um botão de reset, que zera os valores dos registradores. Ele foi utilizado ao longo dos testes das execuções, para facilitar no início de uma nova execução.

Na ULA, as entradas são o registrador ra e o MUX controlado pelo sinal ALUSrc. Ela também é influenciada pelos sinais de controle Branch, Incr e ALUOp e possui duas saídas: o resultado da operação e um sinal Zero, ativado quando resultado = 0. A saída da ULA é enviada ao MUX controlado pelo sinal MemtoReg.

Operações da ULA e ALUOp correspondente:

- soma: 0000
- subtração: 0001
- not: 0010
- and: 0011
- or: 0100
- xor: 0101
- slr: 0110
- srr: 0111
- multiplicação: 1000
- conteúdo de rb: 1001 (utilizado para a instrução mv)

Aqui, como o shifter do logisim aceita no máximo valores de 3 bits para deslocamento (valor armazenado em rb), foi implementado um mux que zera o resultado se o valor de rb for maior que 7 (uma vez que, para um valor a partir de 8, todos os 8 bits serão deslocados).

Código Assembly do trabalho 1:

trabalho-1.asm

Código Assembly, com as instruções novas implementadas:

trabalho-2.asm

Já o circuito em pipeline foi organizado em 5 estágios: busca, decodificação, execução, memória e escrita no banco de registradores. Como a dual-port RAM fornecida possui as saídas de instrução e dados, essas informações foram gerenciadas numa mesma memória. A unidade de controle foi feita em forma de circuito combinacional.

Essa implementação pressupõe que as dependências de dados são resolvidas pelo software, por meio da inserção de “*nops*”. Ainda há alguns ajustes a serem feitos para seu pleno funcionamento, como, por exemplo, o gerenciamento do clock e dos túneis com os dados transportados pelos pipelines.